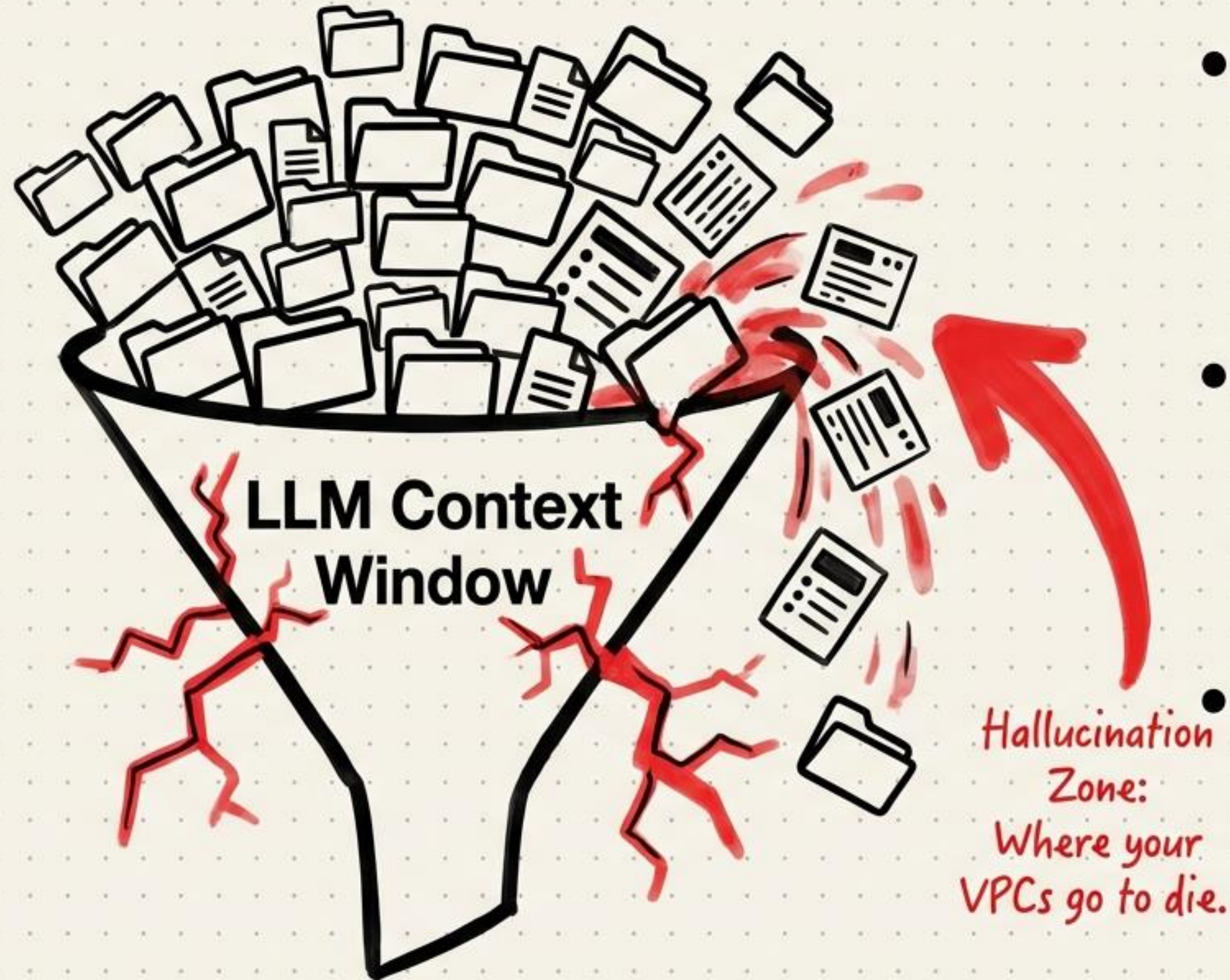


Terraform at Scale: The Copilot Sub-Agent Pipeline

A 5-phase deterministic map-reduce architecture for 30+ repository analysis.

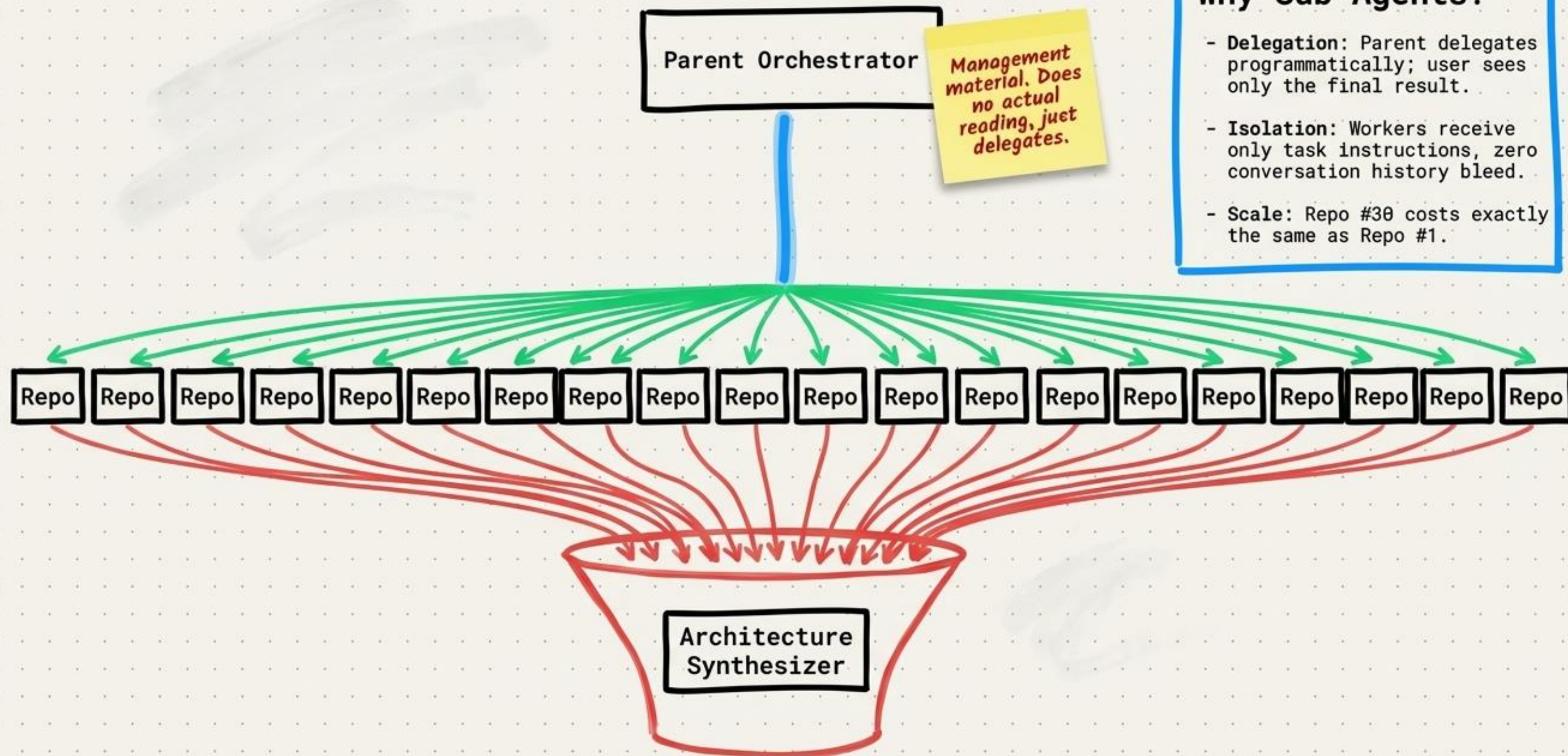
Do not attempt
to read 30
repos yourself.
You will cry.

The Context Collapse



- Tens of thousands of lines of HCL spread across 30+ repositories.
- Cross-repo dependencies cannot be resolved sequentially without memory loss.
- No single context window can hold the full topology simultaneously.

The Map-Reduce Masterplan



Why Sub-Agents?

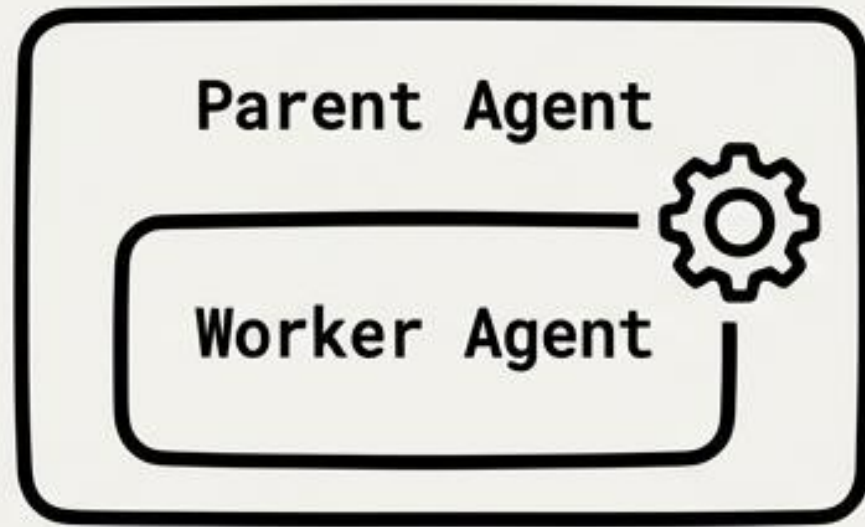
- **Delegation:** Parent delegates programmatically; user sees only the final result.
- **Isolation:** Workers receive only task instructions, zero conversation history bleed.
- **Scale:** Repo #30 costs exactly the same as Repo #1.

Component Translation Matrix

Claude Code	GitHub Copilot	Notes
CLAUDE.md (global rules)	.github/copilot-instructions.md	(Always-on; <=15 rules)
.claude/agents/*.md	.github/agents/*.agent.md	(Sub-agent invocation pattern)
disallowedTools: [Write]	Omit write tools from whitelist	(Copilot is whitelist-only)
tools: [Read, Bash]	tools: ['codebase', 'terminalLastCommand']	Read this twice. No implicit permissions.
.claude/skills/	.github/skills/	(Same open standard)
.claude/settings.json (hooks)	.vscode/settings.json	(Crucial for deterministic blocking)

Pipeline Mechanics: Sub-Agents vs. Handoffs

Sub-Agents (The Pipeline)



- **Traits:** Fully automated. Parent delegates programmatically. Runs in isolated background contexts. Single-level depth.
- **Use Case:** Extraction -> Synthesis
- **Use Case:** Extraction -> Synthesis -> Diagramming -> Analysis.

Handoffs (The Process)



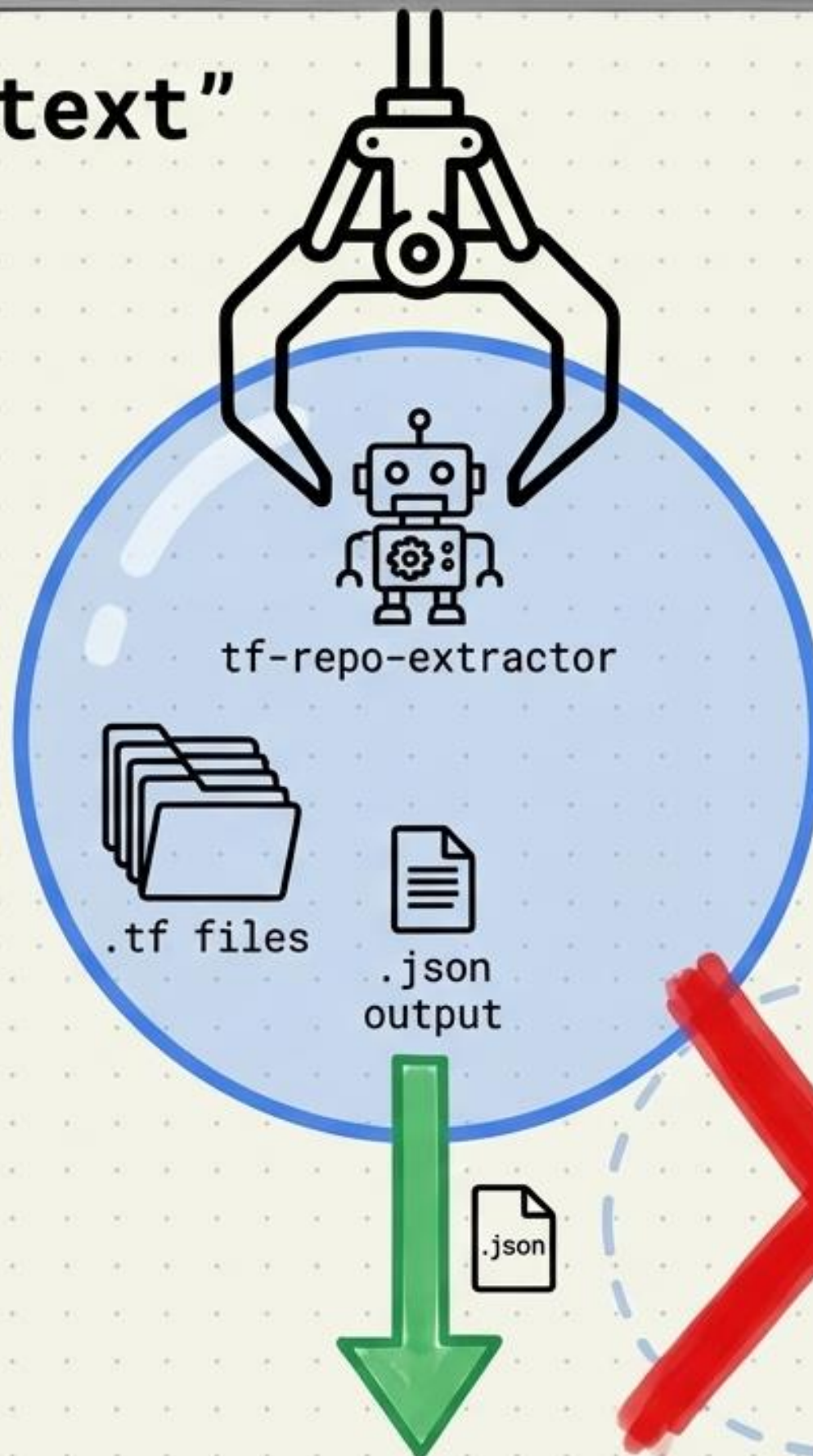
- **Traits:** Sequential. Requires human approval at each checkpoint.
- **Use Case:** Presenting the final parsed report to stakeholders after the pipeline finishes.

Do not use this for data processing unless you enjoy clicking 'Approve' 30 times.

The "Isolated Context" Guarantee

Step 1: Orchestrator drops read-only sub-agent into a sealed context.

Step 2: Sub-agent executes find, grep, cat against local .tf files.

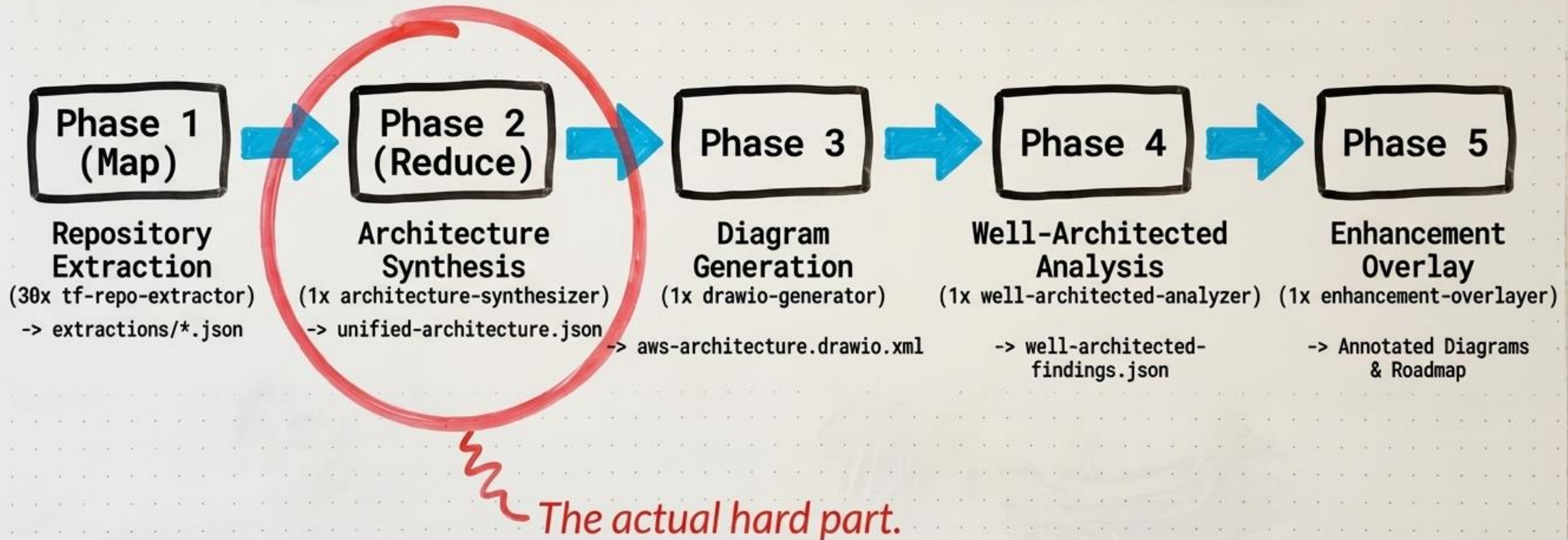


Step 3: Kicks out structured extraction JSON.

Step 4: Bubble pops. Context is completely cleared. No memory accumulation.

Amnesia
as a
Feature.

The 5-Phase Execution Pipeline





Phase 1: Repository Extraction (Map)

Sub-Agent Details:

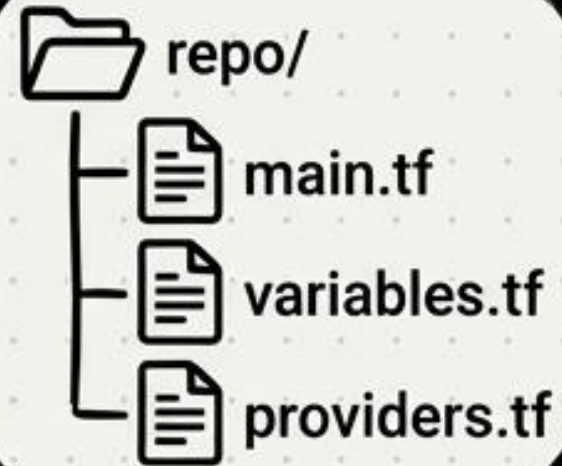
Agent: `tf-repo-extractor` (Read-only capabilities)

Tools: `codebase`, `terminalLastCommand` (standard Unix: `jq`, `python3`, `grep`, `cat`)

Action: Analyzes provider aliases, cross-repo references (e.g., `terraform_remote_state`), and resilience flags (`multi_az=true`).

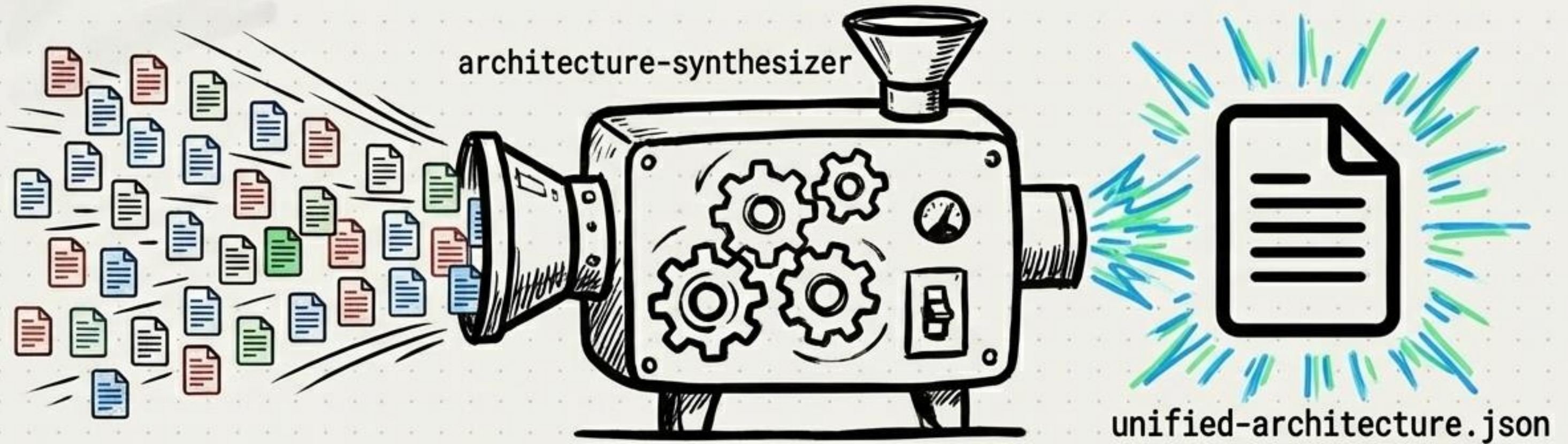
Architectural Rule: Large `.tf` files are parsed by grepping block headers first, then reading specific line ranges, bypassing the need for full-file LLM ingestion.

No 'terraform init' required.
We do it live
(from disk).





Phase 2: Architecture Synthesis (Reduce)



Core Responsibilities

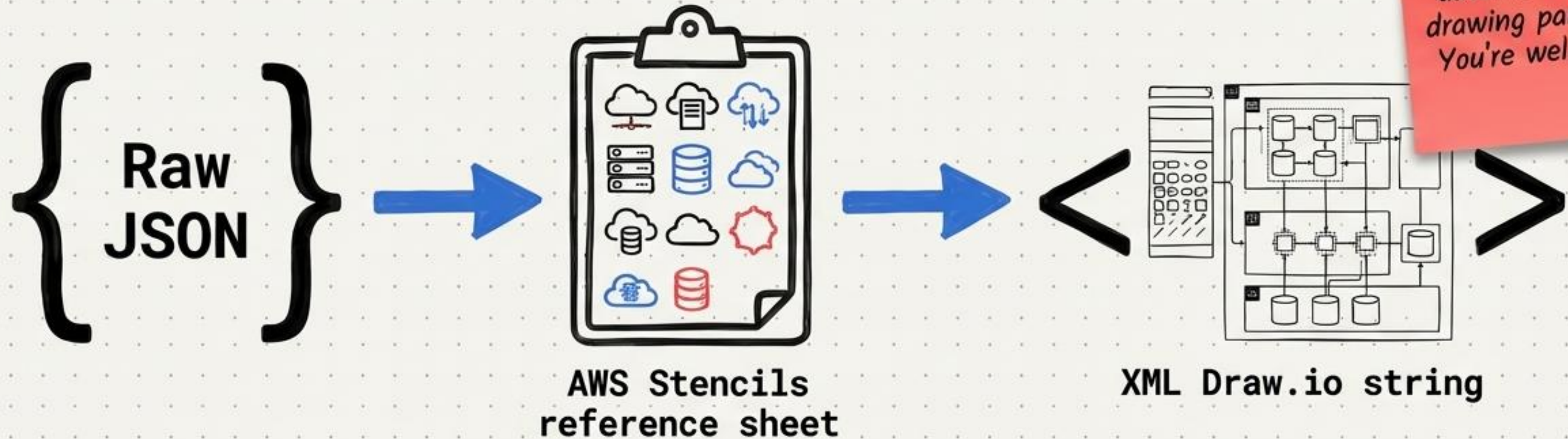
- Resolve cross-repo dependency chains.
- Deduplicate shared resources.
- Build global VPC network topology.
- Trace request flows from edge to data stores.
- Compute blast radius per component.

Constraint Check:
Runs in an isolated context.
Reads compressed,
structured JSON—NOT raw HCL.
This prevents token bloat.



Phase 3: Deterministic Diagraming

Yes, we automated the drawing part too. You're welcome.

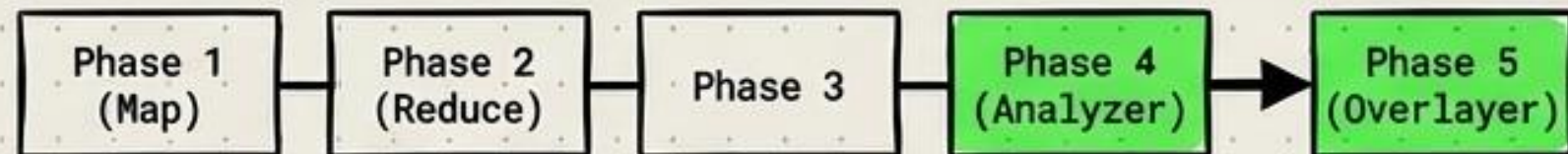


Sub-Agent Details: drawio-generator (Write-capable)

The Mechanism:

- Sub-agent loads aws-stencils.md via Agent Skills only when this phase triggers.
- Generates draw.io XML importable to Lucidchart/Draw.io.

Crucial Cell ID Convention: mxCell id = resource address with dots replaced by dashes. (e.g., aws_rds_cluster.primary -> id='aws-rds-cluster-primary').



Phases 4 & 5: Analysis & Enhancement Overlays

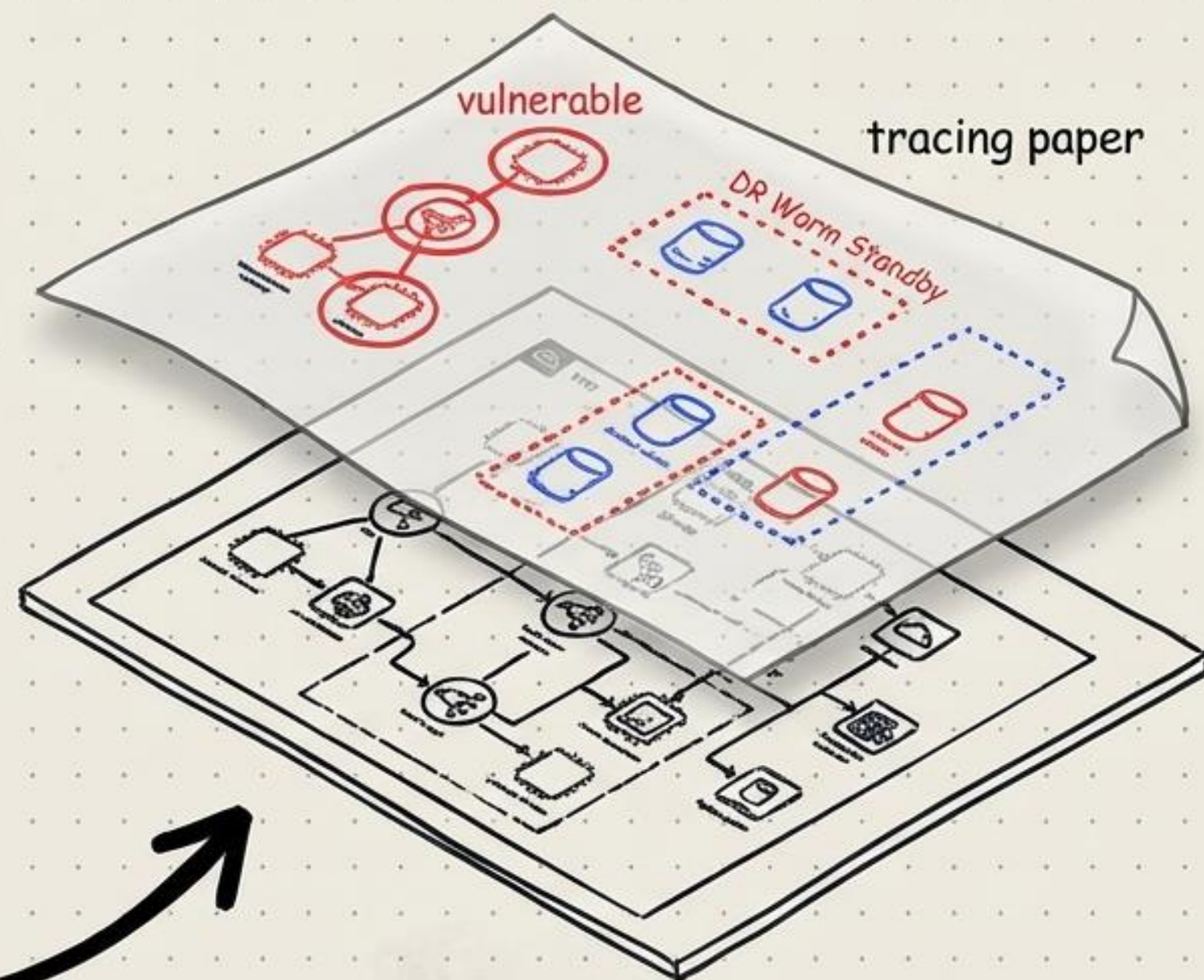
Phase 4 (Analyzer)

- Evaluates unified model against 6 AWS Well-Architected Pillars.
- Scores 1-5, targeting RTO/RPO and single-AZ/single-region failure scenarios.
- Outputs well-architected-findings.json.

Phase 5 (Overlayer)

- Uses the exact mxCell id convention to target specific diagram shapes.
- Injects colored annotations and DR architecture "ghost" overlays directly into the XML.

The budget department's favorite phase.



Guardrails: Vibes vs. Deterministic Rules

~~Vibes / Instructions (Always validate your JSON)~~

- Relies on LLM “best effort”.
- Occasional hallucinations or dropped brackets.
- Costs ~300 tokens every single invocation.

Deterministic Rules / Hooks (validate-extraction-json.sh)

- Executes shell scripts (e.g., jq, python3) at the code level.
- Blocks tools with a non-zero exit code if schema fails.
- Costs 0 tokens. Absolute enforcement.

Stop paying tokens to ask an LLM if a bracket is closed.

The Token Diet (Cost Strategy)

--- HEAVY OPS ---

copilot-instructions.md (Always on):
~300 tokens

Skills & Orchestrator Prompts (Always on):
~1,500 tokens

AWS Stencils (Tier 3 on-demand):
~800 tokens (loaded only in Phase 3)

Sub-agent extractions (30 repos @
~600t each flat): ~18,000 tokens

Synthesis, WAF, Enhancement (Isolated):
~2,000 tokens

--- FREE/OPTIMIZED OPS ---

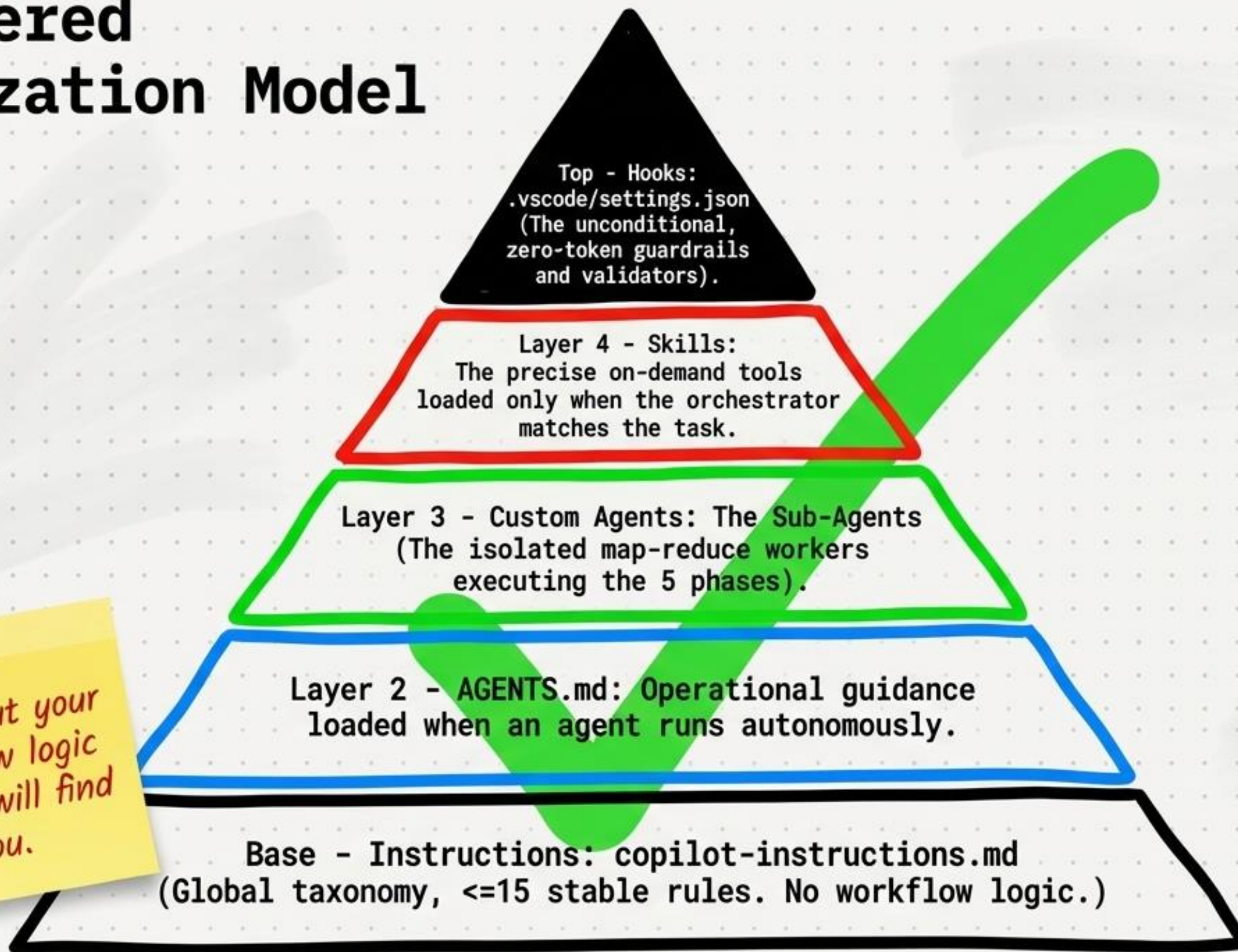
postToolUse Hooks: 0 tokens

TOTAL: ~38,000 tokens



Take that, LLM
pricing model.

The Layered Customization Model



If you put your workflow logic here, I will find you.

The Production-Ready Review

- ☑ Map-Reduce Conquers Scale: 30 repos processed without context collapse via Sub-Agent isolation.
- ☑ Deterministic Schemas: Hooks ensure structured JSON/XML at zero token cost.
- ☑ Pillar-Aligned Synthesis: Raw HCL transformed into a Well-Architected, diagrammed roadmap.

Meeting adjourned.
Go delete
some always-on
instructions.

Closing Callout:

Never always-on what **can be** loaded on-demand.
Never use Instructions when Hooks suffice.

